

Fit simple SECR models

Ian Durbach & Cornelia Oedekoven

Covered in this tutorial

- Load the capture history and mask objects
- Check the covariates available for modelling
- Fit a null SECR model
- Fit simple models with density and detector covariates
- Compare candidate models using AIC
- Save fitted models for the results tutorial

Introduction

In this tutorial we fit a small set of spatial capture-recapture models using `secr.fit()`. The aim is to show the mechanics of fitting models with the objects created in the previous tutorials: the capture history `ch`, the trap covariates attached to `traps`, and the spatial covariates attached to `mask_model`.

An SECR model has a density component and a detection component. The density component describes how expected animal density varies across the mask. The detection component describes how the encounter rate changes with distance from each detector and, optionally, with detector-level covariates.

The models here are intentionally written out explicitly. This makes it clear which parameter is being modelled in each candidate model and avoids hiding the main SECR ideas inside loops or helper functions.

Preparations

We only need the `secr` package for this script because the data preparation has already been done.

```
# Packages needed
library(secr)

output_file <- "tutorials/namkha_basic/output/namkha_basic_fitted_models.RData"
```

Load capture histories and masks

The capture-history file contains the `ch` object and the trap covariates. The mask file contains the full mask, the survey-area mask, and `mask_model`, which is the mask used for model fitting.

```
# Load the capthist object from script 01 and the masks from script 02
load("tutorials/namkha_basic/output/namkha_basic_sec_rinputs_capthist.RData")
load("tutorials/namkha_basic/output/namkha_basic_sec_rinputs_mask.RData")
```

Before fitting models, it is worth checking that the objects contain the expected covariates. Trap covariates can be used in the detection part of the model. Mask covariates can be used in the density part of the model.

```
# Check that the key objects contain the expected covariates
summary(ch)
```

```
Object class      capthist
Detector type     count
Detector number   55
Average spacing   2504.368 m
x-range           534489 576620 m
y-range           3316462 3360989 m
```

```
Usage range by occasion
```

```
      1
min    2
max 251
```

```
Counts by occasion
```

	1	Total
n	26	26
u	26	26
f	26	26
M(t+1)	26	26

```

losses          0      0
detections      47     47
detectors visited 24     24
detectors used   55     55

```

Individual covariates

```

sex
Female : 8
Male   :15
Unknown: 3

```

```
names(covariates(traps))
```

```

[1] "Topography"  "cliff"      "hill"      "valley_stream"
[5] "Water"      "Altitude"

```

```
names(covariates(mask_model))
```

```

[1] "tri"      "d2hydro"  "std_tri"  "std_d2hydro"

```

In the models below:

- **D** is density, modelled using mask covariates such as `std_tri` and `std_d2hydro`
- **lambda0** is the baseline encounter-rate parameter, modelled using trap covariates such as `valley_stream` and `cliff`
- **sigma** controls the spatial scale of detection

Encounter and detection functions

The models use the hazard half-normal detection function, specified with `detectfn = "HHN"`. For count detectors, it is helpful to think in terms of an encounter-rate function: the expected number of encounters declines as the distance between an animal's activity centre and a camera increases.

lambda0 is the encounter-rate parameter at distance zero, and **sigma** controls how quickly encounter rate declines with distance. Both parameters must be positive, so `secr` fits them on a link-function scale. We usually interpret fitted values and predictions on the natural scale rather than directly interpreting the raw coefficients.

Fit candidate models

The fitting mask is `mask_model`, not the Namkha boundary mask. This allows animals with activity centres outside Namkha to contribute to the likelihood if they are still within range of traps near the boundary. Abundance and density can later be summarised over the survey-area mask.

Null model

The null model has no density covariates and no detector covariates. It is the baseline model against which the other candidate models are compared.

```
# The fitting mask is the trap-buffer mask, not the Namkha boundary mask.
# This allows activity centres outside Namkha if they are within range of traps.

# Null model: no density or detector covariates
m0 <- secr.fit(
  ch,
  detectfn = "HHN",
  mask = mask_model,
  model = list(D ~ 1, lambda0 ~ 1, sigma ~ 1),
  verify = FALSE,
  trace = FALSE
)
```

Density covariate models

The next two models allow density to vary with spatial covariates on the mask. `std_tri` is standardised Terrain Ruggedness Index, and `std_d2hydro` is standardised distance to water. Standardised covariates are used because the model coefficients are easier to compare and optimisation is often more stable.

```
# Density varies with ruggedness (TRI)
m1 <- secr.fit(
  ch,
  detectfn = "HHN",
  mask = mask_model,
  model = list(D ~ std_tri, lambda0 ~ 1, sigma ~ 1),
  verify = FALSE,
  start = m0,
  trace = FALSE
)
```

```
)

# Density varies with distance to water
m2 <- secr.fit(
  ch,
  detectfn = "HHN",
  mask = mask_model,
  model = list(D ~ std_d2hydro, lambda0 ~ 1, sigma ~ 1),
  verify = FALSE,
  start = m0,
  trace = FALSE
)
```

The `start = m0` argument gives the optimiser starting values from the null model. This is a practical way to make fitting related models more stable.

Detector covariate model

The next model keeps density constant but allows encounter rate to vary depending on whether the camera was placed in a valley or stream location.

```
# Encounter rate varies depending on whether detection is in a valley or stream
m3 <- secr.fit(
  ch,
  detectfn = "HHN",
  mask = mask_model,
  model = list(D ~ 1, lambda0 ~ valley_stream, sigma ~ 1),
  verify = FALSE,
  start = m0,
  trace = FALSE
)
```

Detector covariates describe camera locations, not animal activity-centre locations. They are therefore used with `lambda0`, not with `D`.

Combined models

The final two models combine density covariates and detector covariates. These models allow spatial variation in density and variation in encounter rate between types of camera sites.

```

# Combined model with a density covariate and a detector covariate
m4 <- secr.fit(
  ch,
  detectfn = "HHN",
  mask = mask_model,
  model = list(D ~ std_tri, lambda0 ~ valley_stream, sigma ~ 1),
  verify = FALSE,
  start = m1,
  trace = FALSE
)

# Combined model with a density covariate and a detector covariate
m5 <- secr.fit(
  ch,
  detectfn = "HHN",
  mask = mask_model,
  model = list(D ~ std_tri + std_d2hydro, lambda0 ~ cliff + valley_stream, sigma ~ 1),
  verify = FALSE,
  start = m4,
  trace = FALSE
)

```

Compare fitted models

AIC compares the candidate models by balancing model fit and model complexity. Lower AIC indicates better support among the models being compared. AIC does not say that a model is true; it ranks the candidate set that we chose to fit.

```

# Compare fitted models using AIC
AIC(m0, m1, m2, m3, m4, m5)

```

							model
m3							D~1 lambda0~valley_stream sigma~1
m5	D~std_tri + std_d2hydro						lambda0~cliff + valley_stream sigma~1
m4							D~std_tri lambda0~valley_stream sigma~1
m0							D~1 lambda0~1 sigma~1
m1							D~std_tri lambda0~1 sigma~1
m2							D~std_d2hydro lambda0~1 sigma~1
	detectfn	npar	logLik	AIC	AICc	dAIC	AICwt
m3	hazard	halfnormal	4	-106.0014	220.003	221.908	0.000 0.4807

```

m5 hazard halfnormal      7 -103.7172 221.434 227.657 1.431 0.2350
m4 hazard halfnormal      5 -105.9684 221.937 224.937 1.934 0.1828
m0 hazard halfnormal      3 -109.1400 224.280 225.371 4.277 0.0566
m1 hazard halfnormal      4 -109.0375 226.075 227.980 6.072 0.0231
m2 hazard halfnormal      4 -109.0967 226.193 228.098 6.190 0.0218

```

For a quick check, we can also inspect the parameter estimates for the model with the lowest AIC.

```

aic_table <- AIC(m0, m1, m2, m3, m4, m5)
best_model_name <- rownames(aic_table)[which.min(aic_table$AIC)]
best_model_name

```

```
[1] "m3"
```

```
summary(get(best_model_name))
```

```
$versiontime
```

```
[1] "5.4.2, run 11:45:51 28 Apr 2026, elapsed 0.62 s"
```

```
$traps
```

```

Detector Number Spacing UsagePct
count          55 2504.368 8247.273

```

```
$scapthist
```

```

Occasions Detections   Animals Detectors   Moves   Animals2
          1          47         26         55         17          12

```

```
$mask
```

```

Cells Spacing Area
2966   1500 667350

```

```
$modeldetails
```

```

CL fixed distribution hcov relativeD
FALSE none      poisson      FALSE

```

```
$AICtable
```

```

              model      detectfn npar    logLik    AIC
D~1 lambda0~valley_stream sigma~1 hazard halfnormal    4 -106.0014 220.003
AICc
221.908

```

```

$coef
              beta    SE.beta        lcl        ucl
D             -8.709311 0.2536294 -9.206416 -8.21220691
lambda0       -5.828055 0.2949014 -6.406051 -5.25005887
lambda0.valley_stream -1.275966 0.6058342 -2.463379 -0.08855296
sigma          8.723669 0.1452827  8.438920  9.00841732

$predicted
      link      estimate SE.estimate        lcl        ucl
D      log 1.650419e-04 4.254176e-05 1.003932e-04 2.713213e-04
lambda0 log 2.943797e-03 8.873510e-04 1.651533e-03 5.247209e-03
sigma   log 6.146687e+03 8.977400e+02 4.623558e+03 8.171578e+03

```

Save fitted models

The fitted model objects are saved for the results tutorial, where they are used to produce AIC tables, abundance estimates, density maps, and covariate-effect plots.

```

save(
  m0, m1, m2, m3, m4, m5,
  file = output_file
)

```

What comes next

The next tutorial summarises these fitted models, estimates abundance and density over the Namkha survey area, maps the predicted density surface, and plots fitted relationships with covariates.